



Security Assessment Report

Marginfi v2 Program

PRs 567 and 585

August 03, 2026

Summary

The Sec3 team was engaged to conduct an incremental review of the PRs 567 and 585 in the Marginfi v2 Program.

The artifact of the audit was the source code of the following programs, excluding tests, in <https://github.com/Odotxyz/marginfi-v2>.

The initial audit focused on the following versions and revealed 8 issues or questions.

#	Task	Type	Commit
P1	PR567: Per bank Circuit breaker guard against oracle attack	Solana	74b24ae → aad4276
P2	PR585: Same asset emode internal	Solana	f05dd16 → 1e33e57

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview	3
Findings in Detail	4
[P1-M-01] Missing circuit breaker price gate in start_liquidation	4
[P1-M-02] Delayed circuit breaker evaluation in direct liquidation	6
[P1-M-03] Risk-carrying instructions accept overvalued collateral	7
[P1-L-01] Bankruptcy accepts unchecked/halted portfolio prices	9
[P1-L-02] Interest freeze is imprecise during a CB halt	11
[P1-L-03] Order execution can bypass the circuit-breaker gate for closed balances	12
[P1-L-04] Circuit breaker checks oracle_price without price_multiplier	15
[P1-I-01] Dust deposits can force liquidations into admin-only mode	19
Appendix: Methodology and Scope of Work	21

Result Overview

Issue	Impact	Status
PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK		
[P1-M-01] Missing circuit breaker price gate in start_liquidation	Medium	Resolved
[P1-M-02] Delayed circuit breaker evaluation in direct liquidation	Medium	Resolved
[P1-M-03] Risk-carrying instructions accept overvalued collateral	Medium	Resolved
[P1-L-01] Bankruptcy accepts unchecked/halted portfolio prices	Low	Resolved
[P1-L-02] Interest freeze is imprecise during a CB halt	Low	Resolved
[P1-L-03] Order execution can bypass the circuit-breaker gate for closed balances	Low	Resolved
[P1-L-04] Circuit breaker checks oracle_price without price_multiplier	Low	Resolved
[P1-I-01] Dust deposits can force liquidations into admin-only mode	Info	Resolved
PR585: SAME ASSET EMODE INTERNAL		
No issues found		

Findings in Detail

PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK

[P1 -M-01] Missing circuit breaker price gate in start_liquidation

Identified in commit [aad4276](#).

The `start_liquidation` instruction initiates a permissionless receivership process for unhealthy margin accounts. However, this entry point contains a critical bypass of the protocol's oracle circuit breaker mechanisms, allowing a liquidator to permanently lock manipulated prices into the liquidation cache for the duration of the multi-instruction receivership flow.

During execution, `start_liquidation` only validates the persisted circuit breaker halt state (via `any_balance_bank_is_cb_halted`) and completely omits the live circuit breaker price gate (`run_cb_price_gate`). It proceeds to fetch live maintenance and equity prices (`HealthPriceMode::Live`) and writes them directly into the locked `liq_cache` and `liq_record.cache`:

```
/* programs/marginfi/src/instructions/marginfi_account/liquidate_start.rs */
043 | check!(
044 |     !any_balance_bank_is_cb_halted(&marginfi_account, ctx.remaining_accounts)?,
045 |     MarginfiError::CircuitBreakerAdminOnly
046 | );
047 | start_receivership(

/* programs/marginfi/src/instructions/marginfi_account/liquidate_start.rs */
092 | pub fn start_receivership<'info>(
102 |     let (_pre_health, assets, liabs) = check_pre_liquidation_condition_and_get_account_health(
103 |         marginfi_account,
104 |         remaining_ais,
105 |         None,
106 |         &mut Some(&mut health_cache),
107 |         HealthPriceMode::Live {
108 |             liq_cache: Some(&mut liq_price_cache),
109 |         },
110 |         ignore_healthy,
111 |     )?;
112 |
113 |     // Use heap-efficient equity calculation
114 |     let (assets_equity, liabs_equity) = get_health_components(
115 |         marginfi_account,
116 |         remaining_ais,
117 |         RequirementType::Equity,
118 |         &mut Some(&mut health_cache),
119 |         HealthPriceMode::Live {
120 |             liq_cache: Some(&mut liq_price_cache),
121 |         },
122 |     )?;
123 |
```

```

124 | write_liquidation_price_cache_from(marginfi_account, remaining_ais, &liq_price_cache)?;
125 | marginfi_account.health_cache = health_cache;
126 | marginfi_account.set_flag(ACCOUNT_IN_RECEIVERSHIP, false);
127 | marginfi_account.indexer_flags.has_ever_been_liquidated = 1;
128 |
129 | // Snapshot values to use in later checks
130 | liq_record.cache.asset_value_maint = assets.into();
131 | liq_record.cache.liability_value_maint = liabs.into();
132 | liq_record.cache.asset_value_equity = assets_equity.into();
133 | liq_record.cache.liability_value_equity = liabs_equity.into();

```

Because the live prices bypass the circuit breaker gate before being permanently cached, a malicious actor can exploit a transient oracle manipulation (e.g., a flash loan attack or brief oracle anomaly) to trigger `start_liquidation` at the exact moment the price is artificially skewed. This locks the anomalous price into the receivership state as the authoritative source of truth.

Once the bad price is locked, the subsequent receivership sub-steps provide no recovery or validation mechanisms to contain the manipulation. The receivership `withdraw` execution explicitly skips the price gate (which is only enforced in the non-receivership branch), the `repay` instruction lacks both the circuit breaker gate and any mechanism to update the cached price, and `end_liquidation` settles the account health strictly on `HealthPriceMode::Cached` without any further CB assertions.

Because the manipulated debt-asset bank is typically only repaid and never withdrawn or pulsed during this process, its circuit breaker is never tripped during the entire receivership.

Resolution

Resolved by commit [38c3eb9](#).

PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK

[P1 -M-02] Delayed circuit breaker evaluation in direct liquidation

Identified in commit [aad4276](#).

The `lending_account_liquidate` instruction provides a permissionless mechanism to liquidate undercollateralized accounts. However, this instruction fails to enforce the `run_cb_price_gate` validation prior to executing the liquidation math, allowing an attacker to consume a manipulated, breaker-breaching oracle price for the first liquidation event before the circuit breaker effectively halts the bank.

During the execution flow, the instruction relies on `update_cache_price` for both the asset and liability banks to refresh the current oracle observations and update the liquidation cache:

```
/* programs/marginfi/src/instructions/marginfi_account/liquidate.rs */
449 | asset_bank.update_bank_cache(group)?;
450 | asset_bank.update_cache_price(asset_price_unbiased)?;
451 |
452 | liab_bank.update_bank_cache(group)?;
453 | liab_bank.update_cache_price(liab_price_unbiased)?;
```

While `update_cache_price` possesses the internal logic to trip the circuit breaker when an anomalous price deviation is detected, this trip mechanism is non-reverting. Because the strict circuit breaker gate (`run_cb_price_gate`) is not explicitly invoked to halt the transaction upon a breach, the instruction proceeds to consume the manipulated price, calculates the discounted asset transfer, and successfully commits the state changes. The bank's internal state is marked as halted, but only *after* the liquidation math has already consumed the anomalous observation.

Resolution

Resolved by commit [38c3eb9](#).

PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK

[P1-M-03] Risk-carrying instructions accept overvalued collateral

Identified in commit [aad4276](#).

Risk-carrying instructions such as `withdraw` (native and third-party integrations like Drift, Kamino, Solend, JupLend), `borrow`, and `order` settlement rely on a portfolio-wide health assessment (`check_account_init_health` / `get_health_components`). However, a critical gap in the protocol's circuit breaker enforcement allows users to leverage overvalued collateral from a bank that has already been halted by the long-window circuit breaker.

During execution, instructions only invoke `validate_bank_state` on the specific bank being acted upon (e.g., the asset being borrowed or withdrawn):

```
/* programs/marginfi/src/instructions/marginfi_account/borrow.rs */
089 | validate_bank_state(&bank, InstructionKind::FailsIfPausedOrReduceState, false)?;

/* programs/marginfi/src/instructions/marginfi_account/withdraw.rs */
078 | let withdraw_is_halt_safe = !marginfi_account.lending_account.has_liabilities();
079 | validate_bank_state(
080 |     &bank,
081 |     InstructionKind::FailsInPausedState,
082 |     withdraw_is_halt_safe,
083 | )?;
```

If a user borrows USDC against a manipulated collateral asset (e.g., a long-tail token), `validate_bank_state` is only checked against the USDC bank. The portfolio health check then values all balances, including the manipulated collateral bank, using `HealthPriceMode::Live`.

While the protocol attempts to protect live prices via `run_cb_price_gate`, this gate specifically evaluates the spot price deviation against the Exponential Moving Average (EMA) at `tier[0]` (short-term deviation). It does *not* explicitly check the bank's persisted `is_cb_halted` flag or evaluate the long-window deviation.

In a "slow-walk" oracle manipulation scenario, an attacker slowly pushes the oracle price upward over time. The EMA naturally follows this slow walk, meaning the spot price remains close to the EMA. Consequently, the live price passes the `cb_price_gate` because the short-term deviation is minimal. The long-window circuit breaker (which compares the current price against a historical anchor) correctly identifies the manipulation and trips, halting the collateral bank.

However, because the halt only restricts direct interactions (deposits/borrows) on the halted bank itself, and `run_cb_price_gate` ignores the long-window halt state, the protocol continues to credit the account

with the overvalued collateral during cross-bank health calculations. This allows an ordinary signer to borrow or withdraw healthy assets against the manipulated collateral that the circuit breaker has explicitly deemed unsafe.

Resolution

Resolved by commit [38c3eb9](#).

PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK

[P1 -L -01] Bankruptcy accepts unchecked/halted portfolio prices

Identified in commit [aad4276](#).

`check_account_bankrupt` reads live prices for all portfolio banks via `HealthPriceMode::Live` to determine insolvency, with no `run_cb_price_gate`. `handle_bankruptcy` calls `validate_bank_state` only on the bankruptcy bank itself (permissionless path `is_halt_safe=false`), so portfolio banks are neither gated nor validated:

```
/* programs/marginfi/src/instructions/marginfi_group/handle_bankruptcy.rs */
057 |     let permissionless_bad_debt_settlement =
058 |         bank.get_flag(PERMISSIONLESS_BAD_DEBT_SETTLEMENT_FLAG);
059 |
060 |     if permissionless_bad_debt_settlement {
061 |         // if permissionless, users can bankrupt reduce-only or operational banks
062 |         validate_bank_state(&bank, InstructionKind::FailsInPausedState, false)?;
063 |     } else {
064 |         // admin can bankrupt banks in any state (CB halt included)
065 |         validate_bank_state(&bank, InstructionKind::Unrestricted, true)?;
066 |         check!(is_admin_or_risk_admin, MarginfiError::Unauthorized);
067 |     }
080 | check_account_bankrupt(
081 |     &marginfi_account,
082 |     ctx.remaining_accounts,
083 |     &mut Some(&mut health_cache),
084 | )?;

/* programs/marginfi/src/state/marginfi_account.rs */
1082 | pub fn check_account_bankrupt<'info>(
1093 |     let (equity_assets, equity_liabs) = get_health_components(
1094 |         marginfi_account,
1095 |         remaining_ais,
1096 |         RequirementType::Equity,
1097 |         health_cache,
1098 |         HealthPriceMode::Live { liq_cache: None },
1099 |     )?;
```

A manipulated or already-halted portfolio bank's price is therefore used in the bankruptcy determination. The settlement amount itself is share-based (`bad_debt = liability_shares × liability_share_value`), so this affects the *trigger* of bankruptcy, not the amount, inflating collateral masks a genuine bankruptcy (bad debt lingers), while deflating collateral / inflating debt false-triggers one (socializing undue loss to depositors). `update_cache_price` updates only the bankruptcy bank's cache, and because `update_circuit_breaker` never errors, its timing relative to settlement is irrelevant, it would not protect the current tx in any position. There is simply no gate on this path, so any halt type or unchecked manipulated price is used.

Resolution

Resolved by commit [38c3eb9](#).

PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK [P1-L-02] Interest freeze is imprecise during a CB halt

Identified in commit [aad4276](#).

The `accrue_interest` halt branch sets `last_update` to the call time, not `cb_halt_ended_at`, and only fires when `accrue_interest` is actually invoked:

```
/* programs/marginfi/src/state/bank.rs */
734 | // Freeze interest accrual during a halt. Deposit/repay remain open while borrow/withdraw
735 | // are blocked, so unfrozen accrual would let new depositors free-ride on borrower
736 | // interest they can no longer escape. Advance `last_update` so post-halt accrual starts
737 | // from the current point instead of catching up on the frozen interval.
738 | if self.is_cb_halted(current_timestamp) {
739 |     self.last_update = current_timestamp;
740 |     return Ok(());
741 | }

/* programs/marginfi/src/state/bank.rs */
907 | fn is_cb_halted(&self, now: i64) -> bool {
908 |     self.get_flag(CIRCUIT_BREAKER_ENABLED) && self.cb_tier > 0 && now < self.cb_halt_ended_at
909 | }
```

The post-halt accrual therefore charges `[last_in_halt_call, halt_ended_at]`, the frozen tail to borrowers/depositors, defeating the freeze's intent.

If no instruction calls `accrue_interest` during the halt (no deposit/repay), the freeze never fires and the entire halt's interest is charged as a lump sum post-halt. `lending_pool_pulse_bank_price_cache` does not call `accrue_interest`, so a pulse-tripped halt leaves `last_update` stale and the first in-halt accrual skips `[last_update, now]` entirely, including any pre-halt pending interest.

Resolution

Resolved by commits [35807ec](#) and [f5a40a4](#).

PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK

[P1 -L -03] Order execution can bypass the circuit-breaker gate for closed balances

Identified in commit [aad4276](#).

The order execution flow only runs the circuit-breaker price gate in `end_execute_order`, after the order's intermediate withdraw and repay instructions have already changed the account balances. The start instruction checks the order trigger with live health components, but it does not run the circuit-breaker gate on the balances whose prices are used to decide whether the order can execute.

```
/* programs/marginfi/src/instructions/marginfi_account/order.rs */
271 | pub fn start_execute_order<'info>(ctx: Context<'info, StartExecuteOrder<'info>>) -> MarginfiResult {
285 |     marginfi_account.set_flag(ACCOUNT_IN_ORDER_EXECUTION, false);
287 |     let (order_assets_in_equity, order_liabs_in_equity, order_asset_count, order_liab_count) =
288 |         get_tagged_account_health_components(
289 |             &marginfi_account,
290 |             ctx.remaining_accounts,
291 |             &order.tags,
292 |         )?;
299 |     let net = order_assets_in_equity
300 |         .checked_sub(order_liabs_in_equity)
301 |         .ok_or_else(math_error!())?;
302 |
303 |     // Check trigger condition
304 |     match order.trigger {
305 |         OrderTriggerType::StopLoss => {
306 |             let sl: I80F48 = order.stop_loss.into();
307 |             check!(net <= sl, MarginfiError::OrderTriggerNotMet);
308 |             order.stop_loss = net.into();
309 |         }
310 |         OrderTriggerType::TakeProfit => {
311 |             let tp: I80F48 = order.take_profit.into();
312 |             check!(net >= tp, MarginfiError::OrderTriggerNotMet);
313 |         }
314 |     }
315 | }
```

While the account is marked as `ACCOUNT_IN_ORDER_EXECUTION`, intermediate withdraw instructions skip the normal health check and the inline circuit-breaker gate. The final cache update may record a breaker trip, but it does not cause the current transaction to fail.

```
/* programs/marginfi/src/instructions/marginfi_account/withdraw.rs */
227 | // Note: during receivership and order execution, we skip all health checks until the end of the transaction.
228 | if !marginfi_account.get_flag(ACCOUNT_IN_RECEIVERSHIP | ACCOUNT_IN_ORDER_EXECUTION) {
229 |     // Check account health, if below threshold fail transaction
230 |     // Assuming `ctx.remaining_accounts` holds only oracle accounts
231 |     // Uses heap-efficient health check to support accounts with up to 16 positions
232 |     check_account_init_health(
233 |         &marginfi_account,
234 |         ctx.remaining_accounts,
235 |         &mut Some(&mut health_cache),
236 |     )?;
242 |     // Inline CB gate: a risk-carrying withdraw (account carries a liability) reverts if
```

```

243 | // any involved bank's live price has jumped past the breach threshold. A
244 | // liability-free withdraw is risk-free and skips the gate.
245 | if marginfi_account.lending_account.has_liabilities() {
246 |     run_cb_price_gate(&marginfi_account, ctx.remaining_accounts)?;
247 | }
248 | }
259 | bank_loader.load_mut()?.update_cache_price(maybe_price)?;

```

The circuit-breaker gate added to the order flow is only run at the end, and it only walks the account balances that are still active at that point.

```

/* programs/marginfi/src/instructions/marginfi_account/order.rs */
343 | pub fn end_execute_order<'info>(ctx: Context<'info, EndExecuteOrder<'info>>) -> MarginfiResult {
389 |     marginfi_account.health_cache = health_cache;
391 |     // Inline CB gate: order execution moves funds at oracle prices, so revert if any involved
392 |     // bank's live price has jumped past the breach threshold relative to its reference.
393 |     run_cb_price_gate(&marginfi_account, ctx.remaining_accounts)?;

/* programs/marginfi/src/state/marginfi_account.rs */
167 | pub fn run_cb_price_gate<'info>(<
168 |     marginfi_account: &MarginfiAccount,
169 |     remaining_ais: &'info [AccountInfo<'info>],
170 | ) -> MarginfiResult<> {
171 |     let clock = Clock::get()?;
172 |     let mut account_index = 0usize;
173 |     for balance in marginfi_account
174 |         .lending_account
175 |         .balances
176 |         .iter()
177 |         .filter(|b| b.is_active())
178 |     {

```

A bank can be used to trigger and constrain an order while its live oracle price would fail the circuit-breaker gate, as long as the execution removes that bank's balance before `end_execute_order` runs.

For example, assume a user has a tagged USDC collateral balance and a volatile SOL liability balance in a stop-loss order. If SOL's live oracle price is manipulated above its circuit-breaker reference, `start_execute_order` can use the inflated liability value to make the order equity look lower and satisfy the stop-loss trigger because no circuit-breaker gate is run there. In the middle of the transaction, the executor can call `lending_account_repay` with `repay_all = true`, closing the SOL liability balance while the account is in order-execution mode. At the end, SOL is no longer an active balance, so `run_cb_price_gate` does not check the SOL bank even though its price was used to trigger the order and to set the start-health baseline.

As a result, order execution can bypass the intended circuit-breaker protection for banks that were active and price-relevant at the start of execution but are removed before the end instruction.

It is recommended to run the circuit-breaker gate at `start_execute_order` for the account's active bal-

ances, so balances used and then closed during execution cannot avoid the price gate.

Resolution

Resolved by commit [b1607ca](#).

PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK

[P1 -L -04] Circuit breaker checks oracle_price without price_multiplier

Identified in commit [aad4276](#).

For oracle setups that combine a base oracle price with an external protocol exchange-rate multiplier, the risk engine uses the multiplier-adjusted price, but the circuit breaker is updated with only the raw base price. This leaves the circuit breaker blind to abrupt changes in the multiplier even though those changes directly affect health, liquidation, order, and bankruptcy price calculations.

The affected cases mainly include the following `OracleSetup` types:

```
/* type-crate/src/types/bank.rs */
271 |   StakedWithPythPush,      // 5
272 |   KaminoPythPush,         // 6
273 |   KaminoSwitchboardPull,  // 7
275 |   DriftPythPull,          // 9
276 |   DriftSwitchboardPull,   // 10
277 |   SolendPythPull,         // 11
278 |   SolendSwitchboardPull,  // 12
279 |   FixedKamino,            // 13
280 |   FixedDrift,             // 14
281 |   JuplendPythPull,        // 15
282 |   JuplendSwitchboardPull, // 16
283 |   FixedJuplend,           // 17
```

These setups follow the same general pattern:

```
base price      = Pyth / Switchboard / fixed price
multiplier      = external integration exchange rate
risk price      = base price * multiplier
CB observation   = base price only
```

The multipliers come from the integration state, not from the base oracle itself:

```
/* programs/marginfi/src/state/price.rs */
188 | fn kamino_price_multiplier(reserve: &MinimalReserve) -> MarginfiResult<I80F48> {
189 |     let (total_liq, total_col) = reserve.scaled_supplies()?;
190 |     if total_col > I80F48::ZERO {
191 |         Ok(total_liq / total_col)
192 |     } else {
193 |         // Note: expected to be unreachable
194 |         Err(MarginfiError::MathError.into())
195 |     }
196 | }

198 | fn drift_price_multiplier(spot_market: &MinimalSpotMarket) -> MarginfiResult<I80F48> {
199 |     let cumulative_interest = u128::from_le_bytes(spot_market.cumulative_deposit_interest);
200 |     Ok(I80F48::from_num(cumulative_interest)
201 |         .checked_div(I80F48::from_num(SPOT_CUMULATIVE_INTEREST_PRECISION)))
```

```

202 |         .ok_or_else(math_error!())?)
203 |     }

205 | fn juplend_price_multiplier(lending: &JuplendLending) -> MarginfiResult<I80F48> {
206 |     Ok(I80F48::from_num(lending.token_exchange_price)
207 |         .checked_div(I80F48::from_num(EXCHANGE_PRICES_PRECISION))
208 |         .ok_or_else(math_error!())?)
209 | }

211 | fn solend_price_multiplier(reserve: &SolendMinimalReserve) -> MarginfiResult<I80F48> {
212 |     let (total_liq, total_col) = reserve.scaled_supplies()?;
213 |     if total_col > I80F48::ZERO {
214 |         Ok(total_liq / total_col)
215 |     } else {
216 |         // Note: expected to be unreachable
217 |         Err(MarginfiError::MathError.into())
218 |     }
219 | }

```

In `OraclePriceWithMultiplier`, the `oracle_price` field is the price before any multiplier is applied, while `price_multiplier` records the rate needed to produce the deposited-token price.

```

/* programs/marginfi/src/state/price.rs */
35 | /// Price per unit before any multipliers are applied, where `price_multiplier` shows what
36 | /// multipliers will be applied to generate the true deposited-token price.
37 | /// * Example: a Staked Collateral bank with an exchange rate of 2 for stake/sol and a price of $50
38 | ///   for SOL will show $50 here, and multiplier will be 2.
39 | /// * For any bank that does not have a multiplier, `price_multiplier = 1` and `oracle_price`
40 | ///   matches the adjusted value.
41 | #[derive(Copy, Clone, Debug)]
42 | pub struct OraclePriceWithMultiplier {
43 |     pub oracle_price: OraclePriceWithConfidence,
44 |     pub price_multiplier: I80F48,
45 | }

```

For example, `OracleSetup::KaminoPythPush` uses two accounts:

```

ais[0] = Pyth oracle account
ais[1] = Kamino reserve account

```

The branch first loads the Kamino reserve and computes the exchange-rate multiplier.

```

/* programs/marginfi/src/state/price.rs */
380 | OracleSetup::KaminoPythPush => {
381 |     // (1) Pyth oracle (for price) and (2) Kamino reserve (for exchange rate)
382 |     check!(ais.len() == 2, MarginfiError::WrongNumberOfOracleAccounts);
384 |     let account_info = &ais[0];
385 |     let reserve_info = &ais[1];
387 |     check_primary_oracle_key(bank_config, account_info)?;
389 |     let reserve_loader = load_kamino_reserve(bank_config, reserve_info)?;
390 |     let reserve = reserve_loader.load()?;
391 |     ensure_kamino_reserve_fresh(&reserve, clock)?;
392 |     let multiplier: I80F48 = kamino_price_multiplier(&reserve)?;

```

At this point:

```
multiplier = Kamino total_liq / total_col
```

The branch then loads the Pyth feed. At this point the feed is still raw Pyth data and has not yet been multiplied by the Kamino rate.

```
/* programs/marginfi/src/state/price.rs */
394 | let mut price_feed =
395 |     PythPushOraclePriceFeed::load_checked(account_info, clock, max_age)?;
```

Before applying the multiplier, the code captures `cache_raw_price`:

```
/* programs/marginfi/src/state/price.rs */
396 | let cache_raw_price = if let Some(price_type) = cache_price_type {
397 |     Some(price_feed.get_price_and_confidence_of_type(price_type, u32::MAX)?)
398 | } else {
399 |     None
400 | };
```

After `cache_raw_price` is captured, the branch applies the multiplier to `price_feed`.

```
/* programs/marginfi/src/state/price.rs */
402 | // Adjust prices & confidence in place
403 | price_feed.price.price = mul_i64_by_i80f48(price_feed.price.price, multiplier)
404 |     .ok_or_else(math_error!())?;
405 | price_feed.ema_price.price =
406 |     mul_i64_by_i80f48(price_feed.ema_price.price, multiplier)
407 |     .ok_or_else(math_error!())?;
408 | price_feed.price.conf = mul_u64_by_i80f48(price_feed.price.conf, multiplier)
409 |     .ok_or_else(math_error!())?;
410 | price_feed.ema_price.conf =
411 |     mul_u64_by_i80f48(price_feed.ema_price.conf, multiplier)
412 |     .ok_or_else(math_error!())?;
```

The later caller then derives two separate outputs from that context:

```
/* programs/marginfi/src/state/price.rs */
905 | let adjusted = context
906 |     .adjusted_price_feed
907 |     .get_price_and_confidence_of_type(oracle_price_type, max_conf)?;
908 | let raw = context.cache_raw_price.unwrap_or(adjusted);
909 | Ok((
910 |     adjusted,
911 |     OraclePriceWithMultiplier {
912 |         oracle_price: raw,
913 |         price_multiplier: context.cache_multiplier,
914 |     },
915 | ))
```

The risk engine uses the adjusted adapter price. For assets and liabilities, health calculation calls `get_price_of_type` on the adjusted `price_feed`, so the value calculation uses the multiplier-adjusted price.

```
/* programs/marginfi/src/state/marginfi_account.rs */
1429 | let lower_price = if let Some(cache) = liq_cache.as_mut() {
1430 |     let price_with_confidence = price_feed.get_price_and_confidence_of_type(
1431 |         requirement_type.get_oracle_price_type(),
1432 |         bank.config.oracle_max_confidence,
1433 |     )?;
...
1437 | price_feed.get_price_of_type(
1438 |     requirement_type.get_oracle_price_type(),
1439 |     Some(PriceBias::Low),
1440 |     bank.config.oracle_max_confidence,
1441 | )?
1442 | };
```

The circuit breaker, however, receives only the raw `oracle_price` from `OraclePriceWithMultiplier`:

```
/* programs/marginfi/src/state/bank.rs */
891 | let clock = Clock::get()?;
892 | self.cache.last_oracle_price = price_with_multiplier.oracle_price.price.into();
893 | self.cache.last_oracle_price_confidence =
894 |     price_with_multiplier.oracle_price.confidence.into();
895 | self.cache.price_multiplier = price_with_multiplier.price_multiplier.into();
896 | self.cache.last_oracle_price_timestamp = clock.unix_timestamp;
897 | self.update_circuit_breaker(
898 |     clock.unix_timestamp,
899 |     clock.slot,
900 |     price_with_multiplier.oracle_price,
901 | )?;
```

If the Kamino multiplier changes while the Pyth base price remains stable, the effective risk price changes but the circuit breaker observes no price movement. For example:

```
raw Pyth price = 100
Kamino multiplier: 1.0 -> 1.5

risk price: 100 -> 150
CB price: 100 -> 100
```

It is recommended to run the circuit breaker against the same effective price used by the risk engine, including the integration multiplier.

Resolution

Resolved by commit [b1607ca](#).

PR567: PER BANK CIRCUIT BREAKER GUARD AGAINST ORACLE ATTACK

[P1-I-01] Dust deposits can force liquidations into admin-only mode

Identified in commit [aad4276](#).

The circuit-breaker liquidation path treats a liquidatee account as admin-only whenever any active balance belongs to a cb_halted bank. This is determined from the liquidatee's remaining accounts, not just the asset and liability banks selected for the current liquidation.

```
/* programs/marginfi/src/instructions/marginfi_account/liquidate.rs */
158 |     let cb_admin_liquidation = any_balance_bank_is_cb_halted(
159 |         &liquidatee_marginfi_account,
160 |         liquidatee_remaining_accounts,
161 |     );
165 |     let signer = ctx.accounts.authority.key();
166 |     if cb_admin_liquidation {
167 |         require!(
168 |             signer == group.risk_admin || signer == group.admin,
169 |             MarginfiError::CircuitBreakerAdminOnly
170 |         );

```

The helper walks every active balance on the liquidatee account and returns `true` if any corresponding bank is currently halted or circuit broken.

```
/* programs/marginfi/src/state/marginfi_account.rs */
127 | pub fn any_balance_bank_is_cb_halted<'info>(
128 |     marginfi_account: &MarginfiAccount,
129 |     remaining_ais: &'info [AccountInfo<'info>],
130 | ) -> MarginfiResult<bool> {
133 |     for balance in marginfi_account
134 |         .lending_account
135 |         .balances
136 |         .iter()
137 |         .filter(|b| b.is_active())
138 |     {
150 |         if bank.is_cb_halted(now)
151 |             || bank.config.operational_state == BankOperationalState::CircuitBroken
152 |         {
153 |             return Ok(true);
154 |         }

```

At the same time, ordinary deposits are marked as halt safe. A user can deposit into a cb_halted bank as long as the bank's effective operational state is not otherwise paused, reduce-only, killed, or uninitialized.

```
/* programs/marginfi/src/instructions/marginfi_account/deposit.rs */
060 | validate_bank_state(&bank, InstructionKind::FailsIfPausedOrReduceState, true)?;

```

This allows a borrower to add a dust position in any compatible cb_halted bank and make that bank an active balance on the account. Subsequent liquidations of the account, even when liquidating unrelated

larger positions, will be routed through the admin-only branch because the liquidatee now has at least one active halted bank.

For example, assume a user has a SOL collateral position and USDC debt, and the account is already eligible for liquidation. If an unrelated compatible bank is `cb_halted`, the user can deposit a minimal amount before a liquidator acts. The dust balance becomes active, so `any_balance_bank_is_cb_halted` returns `true`. A permissionless liquidator attempting to liquidate the SOL/USDC pair will then fail with `CircuitBreakerAdminOnly`, even though the liquidation pair itself does not involve the dust balance.

An admin or risk admin can still liquidate the user's positions by selecting the appropriate `asset_bank`, `liab_bank`, and `asset_amount`. The impact is that an already-liquidatable user can add dust at very low cost to prevent normal permissionless users from liquidating the account, forcing liquidation into an operationally gated admin/risk-admin path and delaying liquidation until that actor intervenes. This can increase bad-debt risk during volatile markets and require risk-admin intervention for accounts that otherwise could have been liquidated permissionlessly.

It is recommended to keep deposits into `cb_halted` banks halt-safe only for accounts that already had an active balance in that bank before the deposit.

Resolution

Resolved by commit [38c3eb9](#).

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and MRGN, Inc. (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

